

physical system — each virtual machine emulating the behaviour of an entire system, complete with its own operating system, storage and I/O, in an isolated fashion. IT infra could now respond more effectively to changes in business requirements, because VMs could be cloned, copied, migrated, and spun up or down to meet demand or conserve resources.

Virtual machines also helped cut costs, because more VMs could be consolidated onto fewer physical machines. Legacy systems running older applications could be turned into VMs and physically decommissioned to save even more money.

But virtual machines still have their share of problems. They are large in size (gigabytes), and each one contains a full operating system. Only some virtualised apps can be consolidated onto a single system. Provisioning a VM still takes a fair amount of time. Finally, the portability of VMs is limited. After a certain point, VMs are not able to deliver the kind of speed, agility and savings that fast-moving businesses are demanding these days.

Containers

Containers work a little like VMs, but in a far more specific and granular way. They isolate a single application and its dependencies — all of the external software libraries the app requires to run — both from the underlying operating system and from other containers. All of the containerised apps share a single, common operating system, but they are compartmentalised from one another and from the system at large.

Taking an example of Docker, if you run `docker run -it ubuntu bash` in your terminal, you can see that like your host OS, your container too, has its own hostname and its own set of processes

What this means is that your Ubuntu container is running in an isolated environment. PID 1 confirms this fact. Similarly, we can provide a mounted storage to our container,

or allocate a particular number of processes or a certain amount of RAM to run with. But what exactly is all this? What exactly is process isolation? What is a containerised environment? What do metered resources mean?

We will try to make sense of all this jargon. We will try to replicate the behaviour of `docker run <image>` as close as possible. To make it all happen, we will be using Go as a language. There is no specific reason behind the selection of Go, though. You can choose any language like Rust, Python, Node, etc, too. The only requirement is that the language should support syscalls and namespaces. I picked Go because I have a personal preference for it. The fact that Docker is built on Go also helps my case.

Building a container from scratch

As mentioned earlier, we will try to replicate something as close to Docker as possible. Just like `docker run <image> cmd args`, we will go for `go run main.go cmd args`. To start with, we will proceed with the basic snippet that most Go plugins of all the major editors have to offer.

```
package main
```

```
func main() {  
  
}
```

Now we will add support for the execution of basic commands like `echo` and `cat`:

```
func must(err error) {  
    // If error exists, panic and exit  
    if err != nil {  
        panic(err)  
    }  
}
```

```
func run() {  
    fmt.Printf("Running %v\n",  
os.Args[2:])  
  
    // Execute the commands that follow  
    'go run main.go run'  
    cmd := exec.Command(os.Args[2],  
os.Args[3:]...)  
    cmd.Stdin = os.Stdin  
    cmd.Stdout = os.Stdout  
    cmd.Stderr = os.Stderr  
  
    must(cmd.Run())  
}  
  
func main() {  
    // Make sure that the first argument  
    after 'go run main.go' is 'run'  
    switch os.Args[1] {  
    case "run":  
        run()  
    default:  
        panic("I'm sorry, what?")  
    }  
}
```

Figure 1 shows what that boils down to.

Now that we can run simple commands with our script, as can be confirmed in Figure 1, we will now try running a bash shell. Since it can get confusing as we are already in a shell, we will try to run `ps` before and after running our script.

Based on Figure 2, it is still difficult to say anything. To confirm if we have isolation like an actual container, let us simply change the hostname from within our bash shell launched using our script. To modify or make any change in the hostname, we need to be a `root`:

The results in Figure 3 mean that we do not have isolation yet. To address this, we need the help of namespaces.

```
lubuntu@lubuntu:~/Projects/make-sense-of-containers$ go run main.go run echo "Hello, world!"  
Running [echo Hello, world!]  
Hello, world!  
lubuntu@lubuntu:~/Projects/make-sense-of-containers$ █
```

Figure 1: Executing the `echo` command in our container